

CSCI1409 Deep Learning | Aidyn Abirov | Week 3 Tasks 1-3 - Estimated time: 1.5-2 hours - **Deadline:** before the next lecture - **Submit:** notebook via MS Teams Assignments

This homework covers Week 3: tracing a small MLP, comparing loss functions, and training a two-hidden-unit regression network using supplied backward-pass formulas.

Rules

- Work individually.
- **Tasks 1-2: no AI** (search engines, lecture notes, textbooks are fine - list references).
- **Task 3: AI allowed** only to explain concepts or help write/debug Task 3 code - not Tasks 1-2. State which tool you used and for what. You must understand and be able to defend every submitted line.
- I or the TA may ask you to explain, trace, or modify code without AI during the practice defense.
- Use Python, NumPy, and Matplotlib only - no autodiff, framework training functions, or scikit-learn estimators.
- Submit Surname_Firstname_Homework03.ipynb via the Assignments tab on Teams; restart the kernel and run all cells first. No submissions once the next lecture starts.

Task 1 - Trace a small MLP

About 25 minutes. No AI.

```
import numpy as np
X = np.array([[1., 0.], [0., 1.], [1., 1.]])
y = np.array([1., 2., 1.])
W1 = np.array([1., -1.], [-1., 1.])
b1 = np.array([0.5, 0.5])
W2 = np.array([1.], [2.])
b2 = np.array([0.])
```

$$Z_1 = XW_1 + b_1, \quad H = \text{ReLU}(Z_1), \quad \hat{Y} = HW_2 + b_2$$

Linear output (regression). Loss: MSE, no factor of one half.

Do:

1. Draw the architecture by hand on paper; take a picture and include it in your notebook when you submit. State the number of hidden layers/units and count all trainable weights and biases, including zeros.
2. Give the shapes of X, y, W1, b1, Z1, H, W2, b2, Yhat.
3. Compute Z1, H, and Yhat by hand for every row; add targets, squared errors, and mean loss to your table.
4. Verify in NumPy. Assert `Yhat.shape == y.shape` before computing residuals.

5. Remove only ReLU, keeping parameters fixed; recompute and explain why the result is affine.
6. Explain why all rows share the same parameters, and why parameter count doesn't depend on the number of examples.

Sanity check: row 1's hidden pre-activations are [1.5, -0.5] (not its activations). Do not train the network in this task.

Task 2 - Choose and compare losses

About 30 minutes. No AI. Use natural logarithms; compute by hand/calculator, then verify in NumPy.

A. Regression - targets [1, 2, 4], predictions [0, 2, 7].

$$MSE = \frac{1}{N} \sum_i (\hat{y}_i - y_i)^2, \quad MAE = \frac{1}{N} \sum_i |\hat{y}_i - y_i|$$

Compute residuals and both losses; repeat with the last prediction changed to 10. Explain the difference in sensitivity to the larger error - the two losses use different scales, so a smaller numeric value doesn't automatically mean "better."

B. Binary classification - targets [1, 0]. Model A predicts [0.8, 0.2]; Model B predicts [0.6, 0.4] (probability of class 1).

$$L = -\frac{1}{N} \sum_i [y_i \ln p_i + (1 - y_i) \ln(1 - p_i)]$$

For each model: mean BCE, predicted classes at threshold $p > 0.5$, and accuracy. Explain how the accuracies can agree while the BCE values differ. Don't feed thresholded classes into BCE.

C. Multiclass - logits [0, ln 3, 0], correct class index 1 (zero-based).

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}, \quad \ell = -\ln p_1$$

Compute the softmax probabilities and the loss; verify they sum to 1. Then, one sentence each: an output representation + loss for (a) an unrestricted real value, (b) a yes/no outcome, (c) exactly one of three categories.

Completion evidence: calculations, NumPy verification, and explanations for A-C. No gradients or training required.

Task 3 - Train a two-hidden-unit MLP

About 50 minutes. AI assistance allowed, with individual defense.

New model and dataset – don't reuse Task 1's arrays. Target $y = |x|$, which bends at zero; two ReLU hidden units can represent its two sides.

```
import matplotlib.pyplot as plt
X = np.array([[ -2.], [ -1.], [ -0.5], [ 0.5], [ 1.], [ 2.]])
y = np.abs(X)
initial = {
    "W1": np.array([[0.5, -0.5]]),
    "b1": np.array([0., 0.]),
    "W2": np.array([[0.5], [0.5]]),
    "b2": np.array([0.]),
}
```

$$Z_1 = XW_1 + b_1, \quad H = \text{ReLU}(Z_1), \quad \hat{Y} = HW_2 + b_2, \quad L = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

A 1-2-1 network. Keep the output linear – no sigmoid or softmax.

A. Forward pass. Implement `forward(params, X)`, returning Z_1 , H , $\hat{Y}$, and `mse(params, X, y)`, returning a scalar. Shapes: Z_1 , H are (6,2); $\hat{Y}$, y are (6,1). Print the initial predictions and mean loss.

Sanity check: at input -2, hidden activations are [0, 1] and the prediction is 0.5. Fix the forward pass if this differs.

B. Backward pass. Implement the supplied gradients (no need to derive general backprop):

$$G = \frac{2}{N}(\hat{Y} - y), \quad dW_2 = H^T G, \quad db_2 = \sum_i G_i$$

$$D = (GW_2^T) \odot \mathbf{1}[Z_1 > 0], \quad dW_1 = X^T D, \quad db_1 = \sum_i D_i$$

The batch-mean factor is already in G – don't divide by N again. Sum with `axis=0` so each bias gradient keeps its bias's shape. ReLU derivative = 0 exactly at zero.

Implement `gradients(params, X, y)`, returning a dict with keys `W1`, `b1`, `W2`, `b2`. Compute all four gradients before changing any parameter, using the same W_2 throughout.

C. Run the experiment.

1. Copy the initial parameters. Print all four gradients at initialization and check that each matches its parameter's shape.
2. Make one simultaneous update, learning rate 0.05. Print the old and newly re-computed loss.

3. Reset every parameter to its initial value. Train for exactly 300 full-batch updates at $lr = 0.05$, saving the loss before and after every update - 301 values total.
4. Plot loss vs. update number (0-300). Print final predictions and final MSE; confirm it's lower than the initial loss.

```
params = {name: value.copy() for name, value in initial.items()}
history = [mse(params, X, y)]
for step in range(300):
    grads = gradients(params, X, y)
    params = {
        name: params[name] - 0.05 * grads[name] for name in params
    }
    history.append(mse(params, X, y))
```

5. Evaluate, without updating parameters, at $X_check = \text{np.array}([[-1.5], [-0.75], [0.75], [1.5]])$ with targets $\text{np.abs}(X_check)$. Print predictions and MSE. Plot the target curve and model predictions on 101 equally spaced points in $[-2, 2]$, marking the training points. (This interpolation check is not evidence of broad generalization.)

Optional gradient check

Only after the required work. Reuse your central-difference approach from Homework 2 ($\text{eps} = 1\text{e-}5$) on one element of W_1 at initialization, keeping every other parameter fixed and using independent parameter copies per perturbation. Compare against the matching entry from gradients. Agreement at that point is evidence, not proof every case is correct - avoid ReLU corners. This is not a fourth required task.

Submission checklist

- All three tasks complete, with explanations and visible outputs
- Task 1: hand-drawn architecture (photographed and embedded), parameter count, shapes, forward table, loss, no-ReLU comparison
- Task 2: regression, binary, and multiclass calculations with NumPy verification
- Task 3: prescribed dataset, initialization, reset, learning rate, and 300 updates
- All parameter gradients match their shapes and are computed before updates
- Loss plot has 301 values; prediction plot and interpolation check included
- External references and any Task 3 AI use recorded; no AI used for Tasks 1-2
- Notebook runs from a fresh kernel; submitted via Teams Assignments before the next lecture begins

Reminder: a working notebook does not replace the individual homework defense during practice.